

CS3810: Introduction to Artificial Intelligence

Mini-Project 1: Search Algorithms for a Cleaning Robot

Assignment Information

Item	Details
Course	CS3810 – Introduction to Artificial Intelligence (Undergraduate)
Assignment	Mini-Project 1
Release Date	Week 4
Progress Check-In	Week 5, Week 6
Due Date	10/02 before class
Total Points	300

Yes, someone made a lot of money with this simple idea, and it was ...

In 1990, three MIT roboticists — Rodney Brooks, Colin Angle, and Helen Greiner — founded a company called iRobot. In 2002, it shipped a \$200 disc that vacuumed floors. The Roomba sold a million units in its first two years, and iRobot has now sold more than 50 million home robots worldwide.

Here is the part that matters for this course: **the original Roomba did not search at all**. It had no map, no plan, and no idea where it was. Rodney Brooks had spent the 1980s arguing against the then-dominant view that a robot should build a model of the world and plan over it; his alternative, *behavior-based robotics*, wired sensors almost directly to motors. So the first Roomba bounced off walls, spiralled outward when it detected dirt, followed edges, and otherwise wandered semi-randomly. Given enough time, a random walk covers a room. It was cheap, it was robust, and it worked well enough to build an industry.

It was also visibly dumb. It re-cleaned the same patch, missed corners, and took far longer than a person would. So the technology moved: by the mid-2010s Roomba models carried a camera and used **visual SLAM** to build a map of the home, and once you have a map, the question changes completely. It is no longer "how do I flail until the floor is clean?" It becomes:

Given a known map, a starting position, and a set of places that need cleaning, what is the cheapest sequence of actions that cleans all of them?

That is a **state-space search problem**, and it is exactly the problem you will solve in this mini-project. You are building the planner for the second kind of Roomba.

Two closing notes. First, this problem is genuinely hard — the general version is closely related to the Traveling Salesman Problem, and you will watch your own search blow up as you add dirty cells. That difficulty is the point of the assignment, not a flaw in your code. Second, engineering is never the whole story: iRobot lost most of its market share to competitors, filed for Chapter 11 in December 2025, and was acquired out of bankruptcy in early 2026. Solving the search problem is necessary. It is not sufficient.

What This Mini-Project Is

This is your first substantial piece of work in CS3810. It is called a *mini-project* rather than a homework because it runs over three weeks, produces a real deliverable, and asks you to make and defend your own design decisions. Expect to spend roughly **15–20 hours** on it. Start in Week 4, not in Week 7.

You will implement three search algorithms — **DFS**, **A***, and **IDA*** — on a grid-world cleaning robot, design and justify heuristics for it, run controlled experiments, and write up what you found.

Working in Pairs

You may work in a pair (exactly two students). If you do:

- Let the instructor know of the team members.
- Each member submit a version of the project in your own code repo
- Include a short **contribution statement** (3–5 sentences) describing who did what. Both partners are responsible for understanding the entire submission.

The Week 6 Check-In

You or your team will submit a one-paragraph status update to the instructor. The point of the check-in is to catch you before you get stuck. Use it.

1. The Problem

1.1 Description

A robot vacuum cleaner operates in a grid-based environment. The robot must visit all dirty cells and clean them while minimizing the total path cost (number of actions). This is the **Vacuum World**, a classic AI testbed introduced by Russell and Norvig, extended here with obstacles and multiple dirty cells.

Extended Vacuum World Example (4x4 grid):

```

+-----+-----+-----+-----+
|  R  |      |  D  |      |
+-----+-----+-----+-----+
|      | ###  |      |  D  |
+-----+-----+-----+-----+
|  D  | ###  |      |      |
+-----+-----+-----+-----+
|      |      |  D  |      |
+-----+-----+-----+-----+

```

R = Robot (starting position)
 D = Dirty cell
 # = Obstacle (impassable)
 = Clean empty cell

Note how much you are being given that a real Roomba is not: a perfect map, exact knowledge of where the dirt is, and a robot that never slips, never gets stuck under a couch, and never runs out of battery. Every one of those assumptions is a research area of its own. Hold onto them — they come back in the reflection question in Section 5.

1.2 Formal Problem Specification

You must implement this specification exactly. Your grader will run your code against grids it supplies, so do not change the interface.

State Representation:

- Robot position: `(row, column)`
- Set of dirty cells remaining
- `State = (position, dirty_set)`

Implementation note: states must be **hashable** so you can store them in a Python `set` or use them as dictionary keys. Use a tuple for position and a `frozenset` for the dirty set: `state = ((row, col), frozenset({(r1, c1), (r2, c2), ...}))` A plain `set` is not hashable and will raise `TypeError` the first time you try to add it to your explored set.

Initial State:

- Robot at the designated starting position
- All dirty cells in the dirty set

Actions:

Action	Effect	Cost
<code>MOVE_UP</code>	Move one cell up	1
<code>MOVE_DOWN</code>	Move one cell down	1
<code>MOVE_LEFT</code>	Move one cell left	1
<code>MOVE_RIGHT</code>	Move one cell right	1
<code>CLEAN</code>	Clean current cell if dirty	1

Transition Model:

- Movement actions change the robot's position if the target cell is valid (in bounds and not an obstacle)
- `CLEAN` removes the current cell from the dirty set if present
- **Invalid actions are treated as illegal and are simply not generated by `get_actions()`.** In particular, do not generate `CLEAN` in a clean cell, and do not generate a move into a wall or obstacle. This keeps your branching factor honest and makes your node counts comparable with your classmates'.

Goal Test:

- The dirty set is empty (all cells cleaned)

Path Cost:

- Sum of action costs (each action costs 1)

2. Part 1 — The Environment (50 points)

Implement a `VacuumWorld` class with exactly this interface:

```

class VacuumWorld:
    """A vacuum-cleaning robot problem on a rectangular grid."""

    def __init__(self, grid, start, dirty):
        """
        Args:
            grid: List of strings or list of lists; '#' marks an obstacle.
            start: Tuple (row, col) - the robot's starting position.
            dirty: Iterable of (row, col) tuples that begin dirty.
        """
        pass

    def initial_state(self):
        """Return the initial state as ((row, col), frozenset(dirty))."""
        pass

    def is_goal(self, state):
        """Return True if the dirty set in `state` is empty."""
        pass

    def get_actions(self, state):
        """Return a list of legal action names available in `state`.

        Legal means: moves stay in bounds and off obstacles, and CLEAN is
        only offered when the robot's current cell is dirty.
        """
        pass

    def result(self, state, action):
        """Return the successor state produced by applying `action`."""
        pass

    def action_cost(self, state, action):
        """Return the cost of `action` in `state` (always 1 here)."""
        pass

```

Requirements:

- Determinism: `get_actions()` must return actions in a fixed order. Use `['MOVE_UP', 'MOVE_DOWN', 'MOVE_LEFT', 'MOVE_RIGHT', 'CLEAN']` as the canonical order and filter it. Two students with the same algorithm should get the same node counts.
- `result()` must **not** mutate the state it is given. Build and return a new one. (This is the single most common bug in this project.)
- Include a `__str__` or `render(state)` method that prints the grid with the robot and remaining dirt marked. You will thank yourself while debugging.

3. Part 2 — The Search Algorithms (150 points)

Implement three algorithms with these exact signatures.

3.1 Depth-First Search (30 points)

```
def dfs_search(problem):
    """
    Perform Depth-First Search.

    Args:
        problem: VacuumWorld instance

    Returns:
        Tuple (solution_path, nodes_expanded, max_frontier_size)
        solution_path: List of actions, or None if no solution
        nodes_expanded: Number of nodes expanded during search
        max_frontier_size: Maximum size of frontier during search
    """
    pass
```

Requirements:

- Use an **iterative implementation with an explicit stack** (not recursion). Recursion will hit Python's recursion limit on the larger test grids.
- Implement cycle detection using an explored set.
- Track metrics: nodes expanded, maximum frontier size.
- Return the **first** solution found (not necessarily optimal).

This is your "original Roomba" baseline: it will find *a* way to clean the room, and it will not be a good one.

3.2 A* Search (50 points)

```
def astar_search(problem, heuristic):
    """
    Perform A* Search.

    Args:
        problem: VacuumWorld instance
        heuristic: Function h(state, problem) -> estimated cost to goal

    Returns:
        Tuple (solution_path, nodes_expanded, max_frontier_size)
    """
    pass
```

Requirements:

- Use a priority queue ordered by $f(n) = g(n) + h(n)$.
- Handle **reopening**: if you reach an already-seen state by a cheaper path, update it in the frontier (or re-insert it). The `PriorityQueue` helper in Section 6.2 supports this.
- Break ties deterministically (the helper's insertion counter does this for you).

- Track metrics as above.
- With an admissible heuristic, your A* must return an **optimal** solution. Test this: `len(atar_path) <= len(dfs_path)` should hold on every grid.

3.3 IDA* Search (70 points)

```
def idastar_search(problem, heuristic):
    """
    Perform Iterative Deepening A* Search.

    Args:
        problem: VacuumWorld instance
        heuristic: Function h(state, problem) -> estimated cost to goal

    Returns:
        Tuple (solution_path, nodes_expanded, iterations)
        iterations: Number of depth-limited iterations performed
    """
    pass
```

Requirements:

- Implement iterative deepening with an **f-cost threshold**. The next threshold is the smallest f-value that exceeded the current threshold.
- Track the number of iterations (threshold increases).
- Use **linear space** — no persistent explored set carried across iterations. You may keep the current path to avoid immediate cycles.
- Return an optimal solution.

Scaffold for IDA*: this is the hardest of the three. The standard shape is a recursive helper `search(path, g, threshold)` that returns either **FOUND**, or the minimum f-value it saw above the threshold, or infinity. Write DFS and A* first; IDA* will make much more sense once you have both.

Why you should care about IDA*: a Roomba's processor has kilobytes of working memory to spare, not gigabytes. A*'s frontier is the thing that kills you on embedded hardware. IDA* trades repeated work for almost no memory — which is exactly the trade a shipping product often has to make.

4. Part 3 — Heuristics (50 points)

Implement and analyze the following heuristics. Each takes `(state, problem)` and returns a number.

Heuristic	Definition
<code>h0</code>	Always returns 0 (this turns A* into uniform-cost search — your baseline)
<code>h1</code>	Number of dirty cells remaining
<code>h2</code>	Number of dirty cells remaining plus the Manhattan distance from the robot to the nearest dirty cell (0 if none remain)

Heuristic	Definition
h3	Your own heuristic — design one that dominates h2 (optional, up to 10 bonus points)

In your report you must:

1. Prove or argue clearly that **h1** and **h2** are **admissible**. Say explicitly what lower bound each one is computing and why the true cost can never be smaller. (Hint for **h2**: obstacles can only make the real path longer than the Manhattan distance, never shorter — and the robot must still clean every dirty cell after it arrives.)
2. State whether **h2** **dominates** **h1**, and what dominance predicts about the number of nodes expanded.
3. If you attempt **h3**, prove admissibility. An inadmissible **h3** that finds shorter paths faster earns no bonus points — it is a different algorithm, not a better heuristic.

5. Part 4 — Experiments and Analysis (50 points)

Run your algorithms on the six test grids provided in `test_grids.py` on the LMS (one 3×3, two 4×4, two 5×5, one 6×6 with a narrow corridor). For each grid × algorithm × heuristic combination, record:

- Solution length (path cost)
- Nodes expanded
- Maximum frontier size (DFS, A*) or iterations (IDA*)
- Wall-clock runtime

Set a 60-second timeout per run. If a configuration times out, report it as **TIMEOUT** — that is a legitimate and interesting result, not a failure.

Present your results as a table, plus at least **two plots** (for example: nodes expanded vs. number of dirty cells, one line per algorithm; and A* nodes expanded under **h0** vs **h1** vs **h2**).

Then answer these questions in your report:

1. DFS usually expands far fewer nodes than A* on some grids, yet returns much worse solutions. Explain both halves of that sentence.
2. How does A*'s node count change as you move from **h0** to **h1** to **h2**? Does this match what dominance predicts? If not, why might it not?
3. IDA* expands *more* nodes than A* on most of these grids. Why would anyone use it anyway? Give a concrete situation where IDA* is the right choice.
4. How does the state space grow as you add dirty cells? Give the formula for the number of reachable states in terms of grid size and number of dirty cells, and explain why this is why search "blows up."
5. Which grid was hardest, and what structural property of that grid made it hard?
6. **The Roomba question.** Section 1.1 listed four assumptions this problem makes that a real robot cannot make (perfect map, known dirt locations, deterministic actions, unlimited battery). Pick **one**, and explain in a paragraph what would break in your implementation if it were removed, and what class of technique would be needed instead. You are not expected to implement it — you are expected to know what kind of problem it becomes.

6. Part 5 — Report and Code Quality (up to 50 bonus points)**6.1 The Report**

Submit a report of **1,200–2,000 words** (excluding tables, figures, and code) containing:

1. **Design decisions** — how you represented states, how you handled reopening in A*, anything you did differently from the spec and why.
2. **Heuristic analysis** — Section 4, items 1–3.
3. **Experimental results** — your tables and plots.
4. **Discussion** — Section 5, questions 1–6.
5. **Reflection** — what broke, how you found it, what you would do differently. Be specific; "I learned a lot about search" earns nothing.

Code quality is graded as part of this section: docstrings on every function, meaningful variable names, no commented-out dead code, and a [README.md](#) saying how to run your experiments.

6.2 Provided Resources

Recommended Libraries

```
# Standard library only for core algorithms
from collections import deque      # For stack/queue
import heapq                       # For priority queue
from typing import Set, Tuple, List, Optional
import time                        # For timing

# For analysis and visualization (required for Part 4 plots)
import matplotlib.pyplot as plt    # For graphs
import pandas as pd                # For tables
```

You may not use a library implementation of DFS, A*, or IDA* (e.g. [networkx.astar_path](#), [simpleai](#), [aima-python](#)). Use of [heapq](#), [deque](#), and [matplotlib](#) is expected and fine.

Priority Queue Helper

You may use this class as-is. It supports decrease-key, which is what makes reopening in A* clean.

```
import heapq

class PriorityQueue:
    """A priority queue with decrease-key functionality."""

    def __init__(self):
        self.heap = []
        self.entry_finder = {}
        self.counter = 0

    def push(self, item, priority):
        if item in self.entry_finder:
            self.remove(item)
        entry = [priority, self.counter, item]
        self.counter += 1
```

```

        self.entry_finder[item] = entry
        heapq.heappush(self.heap, entry)

    def remove(self, item):
        entry = self.entry_finder.pop(item)
        entry[-1] = None # Mark as removed

    def pop(self):
        while self.heap:
            priority, count, item = heapq.heappop(self.heap)
            if item is not None:
                del self.entry_finder[item]
                return item
            raise KeyError('pop from empty priority queue')

    def __contains__(self, item):
        return item in self.entry_finder

    def __len__(self):
        return len(self.entry_finder)

```

Files Provided on the LMS

File	Contents
<code>vacuum_world.py</code>	Stub of the <code>VacuumWorld</code> class with the docstrings above
<code>search.py</code>	Stubs of the three search functions and the four heuristics
<code>priority_queue.py</code>	The helper class above
<code>test_grids.py</code>	The six grading grids, plus the 4×4 example
<code>run_tests.py</code>	Smoke tests — these must pass before you submit
<code>experiments.py</code>	A skeleton harness for Part 4 with timing and timeout handling

7. Deliverables

Submit a single `.zip` named `CS3810_MP1_LastName.zip` containing:

```

CS3810_MP1_LastName/
  vacuum_world.py
  search.py
  heuristics.py
  experiments.py
  results.csv           # Raw Part 4 measurements
  figures/              # Your plots (.png)
  report.pdf            # Sections 1-5 of Part 5
  README.md             # How to run everything
  AI_USE.md             # Required disclosure, see Section 9

```

Before you submit: run `python run_tests.py`. If it fails, your submission is not ready. Graders run it first.

8. Grading Rubric

Component	Points
Part 1: Environment implementation (correct, immutable, deterministic)	50
Part 2a: DFS (iterative, cycle detection, correct metrics)	30
Part 2b: A* (correct, optimal, handles reopening)	50
Part 2c: IDA* (correct, optimal, linear space)	70
Part 3: Heuristics implemented + admissibility arguments	50
Part 4: Experiments, tables, plots, and the six questions	50
Part 5: Report quality, code quality, README	up to 50 bonus points
Total	300
<i>Bonus:</i> admissible <code>h3</code> that dominates <code>h2</code> , with proof and data	+ 10

Partial credit is real. An IDA* that is correct but slow earns most of its 30 points. An IDA* you did not attempt earns zero. Submit what you have.

Common Pitfalls

- Mutating the dirty set inside `result()` — every state in your frontier silently becomes the same state.
- Forgetting that `CLEAN` costs 1, so the optimal cost is at least (moves) + (number of dirty cells).
- Counting "nodes generated" when the spec asks for "nodes expanded." Expanded = popped from the frontier and had its successors generated. Say which one you are counting.
- Letting DFS run without cycle detection. It will not terminate.
- Comparing runtimes on different machines. Compare *node counts* across algorithms; use runtime only as supporting evidence.

9. Academic Integrity

Plagiarism Policy

- All submitted work must be your own
- Any code or text adapted from another source must be cited in a comment or footnote
- Submissions will be checked with plagiarism and code-similarity detection software

Generative AI Policy

This is an AI course, so let's be precise about this.

Permitted:

- Explaining a concept to you (e.g., "what does admissible mean?")
- Helping you debug code **you wrote** — you must be able to explain every line you submit
- Generating boilerplate you understand (plotting code, CSV writing)

- Grammar and clarity editing of report text you wrote

Not permitted:

- Generating your search algorithm or environment implementations
- Generating your heuristics or admissibility arguments
- Generating your analysis, discussion answers, or reflection

Required disclosure: include an `AI_USE.md` listing every AI tool you used, what you used it for, and roughly how much. An honest disclosure costs you nothing. An undisclosed one that surfaces is an integrity violation.

You may be asked to explain any part of your submission to the instructor or TA. Inability to explain your own submission is treated as evidence of a violation, regardless of what the similarity checker says.

Suggested Timeline

Week	Target
Week 5	Read the spec; implement <code>VacuumWorld</code> ; get DFS returning something on the 4×4 example
Week 6	A* working with <code>h1</code> and <code>h2</code> ; check-in ; start IDA*
Week 7	Finish IDA*; run the full experiment grid; make plots; write the report

CS3810 – Mini-Project 1